

Task 03 — Opportunity Closure Validation & Audit Processing

Full walkthrough report: configuration, code, and line-by-line explanation

1. What This Task Asked For

Management wants a business rule for Opportunities: when an Opportunity moves to Closed Won, Salesforce must validate the record, protect sensitive data, and automatically create an audit record. This requires three things working together: a declarative Validation Rule (blocks bad data before it saves), an Apex Trigger (reacts after a successful save), and a supporting Apex class plus a new custom object to store the audit trail.

2. Requirements Checklist

Every control point from the task sheet, matched against what was actually built and tested.

Requirement	Status	Where it lives
Block Closed Won if Amount is empty or zero	Done	Validation Rule: Prevent_Closed_Won_Without_Amount_CloseDate
Block Closed Won if CloseDate is empty	Done	Same Validation Rule (OR condition)
Implemented as a Salesforce Validation Rule (not Apex)	Done	Declarative rule on Opportunity object
Create related audit record on successful Closed Won change	Done	AuditTrigger + AuditEditor.createAuditRecords()
Audit stores Opportunity, Previous Stage, New Stage, Changed Date, Changed By	Done	Opportunity_Audit__c fields
Respect Salesforce security (no assumed field/object access)	Done	with sharing class; trigger's system-mode insert explained in Section 6
Trigger works correctly in bulk (one, many, mixed updates)	Done	Loop over Trigger.new / oldMap, no SOQL or DML inside the loop
Audit failure for one record does not block the whole operation	Done	Database.insert(list, false) with per-record result handling
Explain Order of Execution (validation, trigger, save, failure points)	Done	Section 8 of this report
Manual proof / test evidence	Done	Execute Anonymous run + Opportunity_Audit__c query, Section 9

All 11 control points from the task sheet are covered.

3. The Custom Object: Opportunity_Audit__c

This object exists purely to store a historical snapshot every time an Opportunity becomes Closed Won. It is intentionally simple — no automation of its own, just a record of what changed.

Field API Name	Type	Why it exists
Opportunity__c	Lookup(Opportunity)	Links the audit row back to the Opportunity it describes
Previous_Stage__c	Text(255)	What the Stage was immediately before this change
New_Stage__c	Text(255)	Confirms the Stage the record moved into (Closed Won)
Changed_Date__c	Date	The Opportunity's Close Date at the moment of the change
Changed_By__c	Lookup(User)	Who performed the update, captured via UserInfo.getUserId()

Why a lookup instead of a master-detail? A lookup keeps the audit object independent — if an Opportunity is ever deleted, the audit record is not forced to be deleted with it (master-detail would cascade-delete children). For an audit trail, preserving history even after the parent is gone is usually the safer default.

4. The Validation Rule

Rule name: **Prevent_Closed_Won_Without_Amount_CloseDate**, built on the Opportunity object.

```
AND(  
    ISPICKVAL(StageName, "Closed Won"),  
    OR(  
        ISNULL(Amount),  
        Amount = 0,  
        ISNULL(CloseDate)  
    )  
)
```

Line-by-line:

- **ISPICKVAL(StageName, "Closed Won")** — Stage is a picklist field, not plain text, so it cannot be compared with a plain equals sign in a formula. ISPICKVAL() is the function built specifically for reading a picklist's selected value.
- **AND(...)** — the whole rule only fires when BOTH the stage condition AND the data-problem condition are true. If Stage is anything other than Closed Won, the rule does nothing at all, regardless of what Amount or CloseDate contain.
- **OR(...)** — any one of the three problems below is enough to block the save; they do not all need to be true together.
- **ISNULL(Amount)** — catches a completely empty Amount field.
- **Amount = 0** — catches the case where someone typed 0 instead of leaving it blank. ISNULL() alone would miss this, since 0 is a valid, non-null number.

- **ISNULL(CloseDate)** — catches a missing Close Date.

A note on ISNULL vs ISBLANK: ISBLANK() is meant for text fields and can behave inconsistently on number fields in some contexts, so ISNULL() is the safer, more predictable choice for Amount here.

Error message shown to the user: *"You cannot mark this Opportunity as Closed Won without an Amount and a Close Date."* Error location: Top of Page.

5. The Apex Trigger

File: AuditTrigger.trigger

```
trigger AuditTrigger on Opportunity (after update) {
    if(Trigger.isAfter && Trigger.isUpdate){
        AuditEditor.createAuditRecords(Trigger.new, Trigger.oldMap);
    }
}
```

Line-by-line:

- **trigger AuditTrigger on Opportunity (after update)** — this trigger only runs when an existing Opportunity is updated, and only AFTER Salesforce has already saved it to the database (uncommitted, but saved within the transaction). It never fires on insert or delete.
 - **Trigger.isAfter** and **Trigger.isUpdate** — these are context-check variables that are technically redundant here, since the trigger definition already restricts it to after-update. They are included as a defensive habit: if this trigger definition is ever expanded later to also handle insert/delete, this guard stops the audit logic from running in the wrong context by accident.
 - **Trigger.new** — the list of Opportunity records AFTER the update, exactly as they now exist. Always a List, even when only one record was updated, because triggers must be written to handle bulk operations.
 - **Trigger.oldMap** — a Map of Id to Opportunity showing what each record looked like BEFORE the update. This is the only way to know what the Stage used to be.
 - The trigger body itself does almost nothing — it just hands both collections to AuditEditor.createAuditRecords(). This is a deliberate style choice: triggers should stay thin, and all real logic should live in a testable Apex class.
-

6. The Apex Handler Class

File: AuditEditor.cls

```

public with sharing class AuditEditor {

    public static void createAuditRecords(List<Opportunity> newList, Map<Id,
Opportunity> oldMap){
        List<Opportunity_Audit__c> auditList = new List<Opportunity_Audit__c>();
        Integer auditCount = 0;

        for(Opportunity opp : newList){
            Opportunity oldOpp = oldMap.get(opp.Id);

            if(opp.StageName == 'Closed Won' && oldOpp.StageName != 'Closed Won'){
                Opportunity_Audit__c audit = new Opportunity_Audit__c();
                audit.Opportunity__c = opp.Id;
                audit.Previous_Stage__c = oldOpp.StageName;
                audit.New_Stage__c = opp.StageName;
                audit.Changed_Date__c = opp.CloseDate;
                audit.Changed_By__c = UserInfo.getUserId();

                auditList.add(audit);
                auditCount++;
                System.debug('Audit prepared for: ' + opp.Name);
            }
        }

        if(!auditList.isEmpty()){
            Database.SaveResult[] results = Database.insert(auditList, false);

            Integer successCount = 0;
            Integer failCount = 0;

            for(Database.SaveResult sr : results){
                if(sr.isSuccess()){
                    successCount++;
                }
                else{
                    failCount++;
                    for(Database.Error err : sr.getErrors()){
                        System.debug('Audit creation FAILED: ' + err.getMessage());
                    }
                }
            }

            System.debug('Audit Records Attempted: ' + auditCount);
            System.debug('Audit Records Succeeded: ' + successCount);
            System.debug('Audit Records Failed: ' + failCount);
        }
        else{
            System.debug('No Stage changes to Closed Won detected in this batch.');
```

Line-by-line:

- **public with sharing class AuditEditor** — 'with sharing' means this class respects the running user's record-level sharing rules (org-wide defaults, role hierarchy, sharing rules) whenever it touches data. It does not, by itself, control field-level security — that nuance is covered in the security section below.

- **List<Opportunity_Audit__c> auditList** and **Integer auditCount** — a collection to hold every audit record we plan to insert, plus a counter purely for debug visibility. Nothing is inserted one at a time; everything is batched into this list first.
 - **for(Opportunity opp : newList)** — loops through every updated Opportunity in this transaction, whether that is 1 record or 200. This is what makes the logic bulk-safe: there is no SOQL query or DML statement inside this loop, only in-memory work and list-building.
 - **Opportunity oldOpp = oldMap.get(opp.Id)** — retrieves the pre-update version of this exact record so its old Stage can be compared to the new one.
 - **if(opp.StageName == 'Closed Won' && oldOpp.StageName != 'Closed Won')** — the core rule: only treat this as an auditable event if the Stage is Closed Won NOW and was NOT Closed Won a moment ago. This single condition satisfies three of the task's trigger-behavior requirements at once: it fires once per Opportunity that is updated, it is unaffected by how many Opportunities are updated together, and it ignores any edit that leaves Stage untouched or that was already Closed Won before this save.
 - **audit.Opportunity__c = opp.Id** through **audit.Changed_By__c = UserInfo.getUserId()** — builds one audit row per matching Opportunity, capturing the four required data points from the task sheet: which Opportunity, its previous stage, its new stage, and who made the change (via the running user's Id).
 - **auditList.add(audit)** — queues the record instead of inserting immediately, so the whole batch can be inserted together in one DML statement later. Doing DML inside a loop is one of the most common beginner mistakes in Apex and can hit governor limits quickly in bulk scenarios; this code avoids that entirely.
 - **Database.insert(auditList, false)** — this is the key line for requirement 5 (failure handling). A plain 'insert auditList;' statement throws an unhandled DmlException the moment ANY record in the list fails, and since a trigger runs inside the same transaction as the Opportunity save, that exception would roll back the Opportunity update too — even though it had already passed validation. Passing 'false' as the second argument switches to partial-success mode: Salesforce attempts every record, and hands back a result array instead of throwing.
 - **Database.SaveResult[] results** and the for-loop over it — walks through each individual result. `sr.isSuccess()` tells us whether that specific audit record was saved. If not, `sr.getErrors()` gives the actual reason (e.g. required field missing, no access), which is written to the debug log rather than surfaced as a user-facing error.
 - The final three `System.debug` lines are purely for visibility while testing — the same convention used in the AccountCustomize class from the previous task, giving a clear before/after count of what happened.
-

7. Security Handling

- **with sharing** on the class means the trigger's DML respects org-wide defaults and sharing rules for records — it will not let a user's automation reach into records they are not shared with.
- **Apex trigger context runs in system mode by default.** This means the trigger itself bypasses field-level security (FLS) and object CRUD permissions on Opportunity_Audit__c, regardless of whether the sales rep who triggered the update actually has Create access on that object. This is intentional and correct for an audit trail: the audit log is a system-level control, not a feature the end user opts into, so it should not silently fail just because a rep's profile was never given access to the audit object.
- The flip side of that same fact is why requirement 3 ('do not assume every user has access to every field') matters: nothing in this design reads Opportunity fields back out to the user through a report or UI component, so no extra field-level check was needed there. If this logic were ever extended to display audit data back to end users (e.g. a related list), that display layer would need its own FLS-aware query (WITH SECURITY_ENFORCED or Security.stripInaccessible), separate from this insert logic.

- The Validation Rule itself is enforced for every user and every save path (UI, API, Data Loader, flows) with no exceptions except System Administrators who choose to bypass validation — there is no way for a lower-privilege user to skip it.
-

8. Salesforce Order of Execution — Applied to This Scenario

This is the sequence Salesforce actually follows when a user changes an Opportunity's Stage to Closed Won and clicks Save:

- 1. The edited values are loaded over the original record in memory.
- 2. Salesforce runs its own basic system validation (data types, required standard fields).
- 3. Any **before** triggers on Opportunity would run here — this task has none.
- 4. **Custom Validation Rules run next** — this is where Prevent_Closed_Won_Without_Amount_CloseDate is evaluated. It runs after before-triggers but before the record is written to the database.
- 5. If the Validation Rule's condition is true, the save is rejected immediately: the user sees the red error banner, nothing is written to the database, and execution stops here completely — no triggers fire at all.
- 6. If the Validation Rule passes, the record is saved to the database (given an Id / updated) as part of the still-open transaction.
- 7. **After triggers run** — this is when AuditTrigger fires, calling AuditEditor.createAuditRecords(). At this point the Opportunity's Closed Won change already exists in the transaction.
- 8. Any remaining automation (workflow rules, Flow, assignment rules) would run after this.
- 9. Only once every step in the transaction finishes without an unhandled error does Salesforce commit the whole transaction to the database permanently.

Answers to the specific questions the task sheet asked:

- **When does the Validation Rule execute?** After any before-triggers, but before the record is saved and before after-triggers run.
 - **When does the trigger fire?** After the Validation Rule has passed and the Opportunity has been saved within the transaction — since AuditTrigger is an after-update trigger.
 - **When is the Opportunity actually saved?** Immediately after validation passes and before after-triggers run. That save is only made permanent once the entire transaction (including the trigger's audit insert) completes without an unhandled error.
 - **What happens if the Validation Rule fails?** The whole save is aborted right there. The user sees the error message, nothing changes in the database, and the trigger never runs.
 - **What happens if an exception is thrown during the transaction?** An unhandled exception rolls back everything in that transaction, including an Opportunity save that had already happened moments earlier. This is exactly why AuditEditor uses Database.insert(list, false) instead of a plain insert — it turns a potential unhandled exception into a handled result, so a failed audit record cannot undo an already-valid Opportunity update.
-

9. Test Evidence From This Session

An Execute Anonymous test was run against an existing Opportunity ('Dickenson Mobile Generators', starting Stage: Qualification) using this script:

```

Opportunity opp = [SELECT Id, StageName, Amount, CloseDate FROM Opportunity
                    WHERE StageName != 'Closed Won' AND Amount != null
                    AND Amount != 0 AND CloseDate != null LIMIT 1];
System.debug('Testing with: ' + opp.Id + ' current stage: ' + opp.StageName);
opp.StageName = 'Closed Won';
update opp;

```

Resulting debug log:

```

Testing with: 006dM00000TPQ6fQAH current stage: Qualification
Audit prepared for: Dickenson Mobile Generators
Audit Records Attempted: 1
Audit Records Succeeded: 1
Audit Records Failed: 0

```

This confirms the full chain worked end to end: the Opportunity passed validation (it had a valid Amount and CloseDate), the trigger fired after the save, the handler correctly identified the Stage change into Closed Won, and the audit insert succeeded with zero failures.

Two follow-up checks were set up to complete the evidence trail: querying Opportunity_Audit__c directly to confirm the row's field values match (Previous_Stage__c = Qualification, New_Stage__c = Closed Won), and re-attempting a Closed Won save with a blank Amount on a different record to confirm the Validation Rule blocks it and that no extra audit row gets created for the rejected save.

10. How To Reason Through Similar Problems Next Time

- **Start by separating "prevent bad data" from "react to good data."** Anything that should stop a save belongs in a Validation Rule (or a before-trigger, for more complex logic). Anything that should happen only after a save succeeds belongs in an after-trigger.
- **Ask what changed, not just what the record looks like now.** Any time a trigger only cares about a transition (like Stage moving INTO Closed Won, not just being Closed Won), you need oldMap or Trigger.oldMap to compare before vs. after — which only exists on update triggers.
- **Never put a SOQL query or a DML statement inside a for-loop.** Build lists during the loop, then query or insert/update once, outside the loop, for the whole batch. This is the single most common thing beginner Apex code gets wrong.
- **Decide on purpose whether a secondary failure should block the primary operation.** Ask: is this secondary record a hard requirement, or a supporting/logging record? If it's supporting (like an audit trail), use Database.insert(list, false) and handle failures gracefully. If it's a hard requirement, let it throw and roll back on purpose.
- **When asked about Order of Execution, always anchor your answer to a concrete sequence:**
before-triggers → validation rules → duplicate rules → save (assigns Id) → after-triggers → workflow/flow → final commit. Map whatever the question asks about onto this sequence rather than trying to memorize it as a separate fact for every scenario.